



Next-Level Logistics

Farmer Data Collection System · Handover & Documentation

Deliverable 1

Tool	Farmer Data Collection System
Client	Circulaire Stromen - Next Level Logistics (LCB / Midpoint Brabant)
Institution	Breda University of Applied Sciences (BUas)
Author	Marin Chiosa · Data Science & AI
Prepared for	LCB and whoever continues the work · June 2026 · v1.0

[!IMPORTANT] **Before going to production**, change all default passwords listed above. The MinIO and PostgreSQL credentials in `docker-compose.yml` are development defaults and must not be used in production.

1.2 GitHub Repository

Repository: `Next-Level-Logistics`

Folder Structure

```
Next-Level-Logistics/
├── api/                                # FastAPI backend (Python)
│   ├── app/
│   │   ├── core/                      # Core configuration
│   │   ├── db/                        # Database session management
│   │   ├── models/                   # SQLAlchemy ORM models (models.py)
│   │   ├── schemas/                 # Pydantic request/response schemas
│   │   ├── routers/                 # API endpoint handlers
│   │   │   ├── records.py            # Harvest data CRUD
│   │   │   ├── cargo_offers.py       # Cargo marketplace endpoints
│   │   │   ├── infrastructure.py     # Farms, farmers, locations
│   │   │   ├── uploads.py            # Photo upload handling
│   │   │   └── rdw_vehicles.py        # Dutch vehicle registry lookup
│   │   ├── services/                # Business logic services
│   │   │   ├── gemini_vision.py       # Google Gemini AI integration
│   │   │   ├── google_maps_service.py # Maps route caching
│   │   │   ├── push_to_danil.py       # Real-time push to partner
│   │   │   └── webhook.py             # Outbound webhook on confirm
│   │   └── pipeline/                 # AI processing pipeline
│   │       ├── processor.py           # Main AI orchestrator
│   │       ├── vision.py              # Vision model abstraction
│   │       └── ocr.py                 # OCR text extraction
│   ├── Dockerfile                    # Production container definition
│   ├── requirements.txt               # Python dependencies
│   ├── seed.py                       # Database seeder (real farms)
│   ├── farmer_data.db                 # SQLite database (local dev)
│   └── uploads/                       # Local photo storage
├── app/                              # React Native mobile app (Expo)
│   ├── App.js                        # Root component & navigation
│   └── src/
│       ├── screens/                  # UI screens
│       │   ├── LoginScreen.js
│       │   ├── HomeScreen.js
│       │   ├── SubmissionScreen.js    # Main harvest data entry
│       │   ├── HistoryScreen.js
│       │   ├── LicensePlateScreen.js  # RDW vehicle lookup
│       │   ├── HelpScreen.js
│       │   ├── MyCargoOffersScreen.js # Cargo marketplace
│       │   ├── CargoOfferCreationScreen.js
│       │   ├── AvailableCargoScreen.js
│       │   ├── CargoRouteMapScreen.js
│       │   └── CargoBookingScreen.js
```

```

| | | └─ services/           # API clients & utilities
| | |   └─ api.js            # Main API client (Axios)
| | |   └─ cargoOfferService.js # Cargo API client
| | |   └─ rdwService.js      # RDW vehicle API client
| | |   └─ googleMapsService.js # Maps deep links
| | |   └─ feedbackService.js  # In-app feedback
| | |   └─ locationService.js   # GPS & location
| | |   └─ offlineQueue.js      # Offline submission queue
| | |   └─ userSession.js        # Session management
| | |   └─ sampleData.js         # Demo/fallback data
| | | └─ components/           # Reusable UI components
| | | └─ theme/                # Design system (colors, typography, spacing)
| | | └─ utils/                # Accessibility & responsive helpers
| | | └─ constants/            # App-wide constants
| | └─ package.json
| └─ app.json                  # Expo configuration
|
└─ dashboard/                 # React web dashboard (MUI)
  └─ src/
    └─ App.js                  # Main dashboard with tabs
    └─ components/
      └─ DashboardComponents.js # All dashboard UI
      └─ services/
    └─ Dockerfile
    └─ package.json
  |
└─ docker/                    # Docker Compose configurations
  └─ docker-compose.yml        # Dev (PostgreSQL, MinIO, Redis)
  └─ docker-compose.prod.yml   # Production (API, Dashboard, DB, Redis)
  |
└─ docs/                      # Project documentation
  └─ FARMER_DATA_SYSTEM_README.md # Full system specification
  └─ DEPLOY.md                  # Deployment guide
  └─ IMPLEMENTATION_SUMMARY.md   # Implementation details
  └─ ACCESSIBILITY_TESTING.md     # WCAG 2.1 AA testing
  └─ RESPONSIVE_DESIGN.md         # Responsive design specs
  └─ USER_TESTING_GUIDE.md        # User testing framework
  └─ PHASE1_COMPLETE.md           # Phase 1 completion notes
  └─ README.md                    # Deployment quick reference
  |
└─ .env.example                # Environment variable template
└─ .env                        # Root environment (Docker)
└─ start.bat                   # Windows one-click launcher
└─ IMPLEMENTATION_GUIDE.md      # Cargo marketplace feature guide
└─ .gitignore

```

1.3 Server Details & URLs

Component	URL / Port	Notes
API Backend	<code>http://localhost:8000</code>	FastAPI with auto-docs at <code>/docs</code>
Dashboard (dev)	<code>http://localhost:3000</code>	React dev server

Component	URL / Port	Notes
Dashboard (prod)	<code>http://localhost:80</code>	Served via Docker / also mountable from API
University Server	<code>http://194.171.191.226:3061</code>	Production deployment target
Mobile App	Expo Go (scan QR)	No fixed URL — connects to API
Danil's endpoint	<code>https://sfsc-bot.duckdns.org/marin/upload</code>	Real-time data push partner
RDW Vehicle API	<code>https://opendata.rdw.nl</code>	Dutch vehicle registry (public, no key needed)

1.4 Third-Party Services

Service	Purpose	Auth method
Google Gemini 2.5 Flash	AI vision analysis of farm produce photos	API key in <code>api/.env</code>
RDW Open Data	Vehicle information lookup by license plate	Public API, no key
Google Maps / Waze	Navigation deep links in cargo marketplace	Deep links only, no API key required for links

1.5 How to Run, Deploy, and Restart

Local Development (Quick Start)

Option A — One-click (Windows):

```
# Double-click or run:
start.bat
```

This kills existing processes on ports 8000/3000, starts the API server, then the dashboard dev server.

Option B — Manual:

```
# Terminal 1: Start API
cd api
pip install -r requirements.txt
python -m uvicorn app.main:app --reload --host 0.0.0.0 --port 8000

# Terminal 2: Start Dashboard
cd dashboard
npm install
npm start

# Terminal 3: Start Mobile App
cd app
# IMPORTANT: Create a .env file in the app/ directory with your local network IP:
```

```
# echo EXPO_PUBLIC_API_URL=http://YOUR_LOCAL_IP:8000/v1 > .env
npm install
npx expo start
# Scan QR code with Expo Go app on your phone
```

Docker Development

```
# Start supporting services (PostgreSQL, MinIO, Redis)
cd docker
docker-compose up -d

# Then start API and dashboard manually (see above)
```

Production Deployment

```
# Build and deploy all services
docker-compose -f docker/docker-compose.prod.yml up -d --build

# Initialize the database
docker exec -it farmer_api_prod python /app/seed.py

# Check status
docker-compose -f docker/docker-compose.prod.yml ps

# View logs
docker-compose -f docker/docker-compose.prod.yml logs -f api
```

Restart Procedure

```
# Restart all production services
docker-compose -f docker/docker-compose.prod.yml restart

# Or restart a specific service
docker-compose -f docker/docker-compose.prod.yml restart api

# Full rebuild (after code changes)
docker-compose -f docker/docker-compose.prod.yml down
docker-compose -f docker/docker-compose.prod.yml up -d --build
```

Database Management

```
# Seed initial data (farms, farmers, locations)
cd api
python seed.py

# Reset database
python reset_db.py

# Check database contents
python check_db.py
```

```
# Backup (production)
docker exec farmer_db_prod pg_dump -U user farmer_data > backup.sql
```

2. Functional Design

2.1 What Does the Tool Do?

The **Farmer Data Collection System** is a mobile-first platform that enables farmers in short food supply chains to log harvest, condition, and transport data **passively** — through photos and a minimal 5-field form — without requiring active data entry effort. An AI pipeline (Google Gemini 2.5 Flash) analyses photos of produce crates to automatically extract product type, quantity estimates, and condition ratings. All collected data feeds into a central logistics dashboard for supply chain managers.

Additionally, a **Cargo Marketplace** feature allows farmers to share available truck capacity, browse other farmers' cargo offers, view routes, and coordinate shared transport.

2.2 Who Is the Intended User?

User type	Description	Primary interface
Farmers	Small-scale agricultural producers with basic smartphones, low technical literacy, and no tolerance for complex workflows. Primary language: Dutch.	React Native mobile app
Supply chain managers	Logistics coordinators at hubs (e.g., Brabants Streekgoed, The Food Directors) who need to review, confirm, and export harvest data.	Web dashboard
Partner systems	External logistics management systems (e.g., Danil's WhatsApp bot) that consume data via webhooks and push APIs.	API / webhook

2.3 What Problem Does It Solve?

Short food supply chains lack logistical data at the farm level. Farmers are not data workers — they cannot manually enter structured records. This system solves the data gap by:

1. Meeting farmers where they are (a smartphone photo or 5-field form)
2. Using AI to pre-fill data fields from photos, so the farmer **confirms** rather than **types**
3. Flowing structured data automatically to logistics management systems
4. Enabling collaborative transport through the cargo marketplace

2.4 Main User Flow — Step by Step

Flow A: Harvest Data Submission (Farmer)

1. Open app → Login screen
 - └ Enter farmer username (e.g., "henk_van_den_berg")
2. Home screen
 - └ Tap "Log a Harvest" → Submission screen
3. Submission screen
 - └ [Take Photo] → Camera opens → Photo captured
 - └ Photo uploaded to server → AI analyses it
 - └ Form fields pre-filled automatically
 - └ Low-confidence fields highlighted in amber
 - └ [Fill Manually] → Empty form displayed
4. Form fields (5 required):
 - └ Product type (dropdown, e.g., "Tomatoes")
 - └ Quantity (number input, e.g., 50)
 - └ Unit (dropdown: Crates / Kg / Units / Boxes)
 - └ Condition (3-chip tap: Good / Mixed / Damaged)
 - └ Location (dropdown of predefined farm locations)
 - + Notes (optional text, max 200 chars)
5. Tap "Submit" → Record saved → Pushed to partner systems
 - └ Return to Home screen → "My Records" shows history

Flow B: Cargo Marketplace (Farmer)

1. Tap 🚚 Cargo tab in bottom navigation
2. "My Offers" screen
 - └ Tap + Create New Offer
 - └ Step 1: Enter license plate (e.g., "AB-12-CD")
 - └ RDW API fetches vehicle details & cargo capacity
 - └ 20% capacity reduction applied automatically
 - └ Step 2: Fill delivery details
 - └ Destination location & coordinates
 - └ Delivery date & time window
 - └ Contact phone (optional)
 - └ Submit → Offer published to marketplace
3. "Browse" screen – View all active cargo offers
 - └ Tap any offer → Route Map screen
 - └ Vehicle & driver info
 - └ Route distance & duration
 - └ Navigate via Waze / Google Maps deep links

Flow C: Dashboard (Supply Chain Manager)

1. Open `http://localhost:8000` (or `:3000` for dev)
2. Dashboard tab
 - └ Stats cards: Total records, confirmed %, flagged count
 - └ Records table with filters (status, date range)

- └ Click any row → Detail panel opens
 - └ View photo, extracted fields, confidence scores
 - └ Confirm / Flag / Reject the record

3. Transport Hub tab

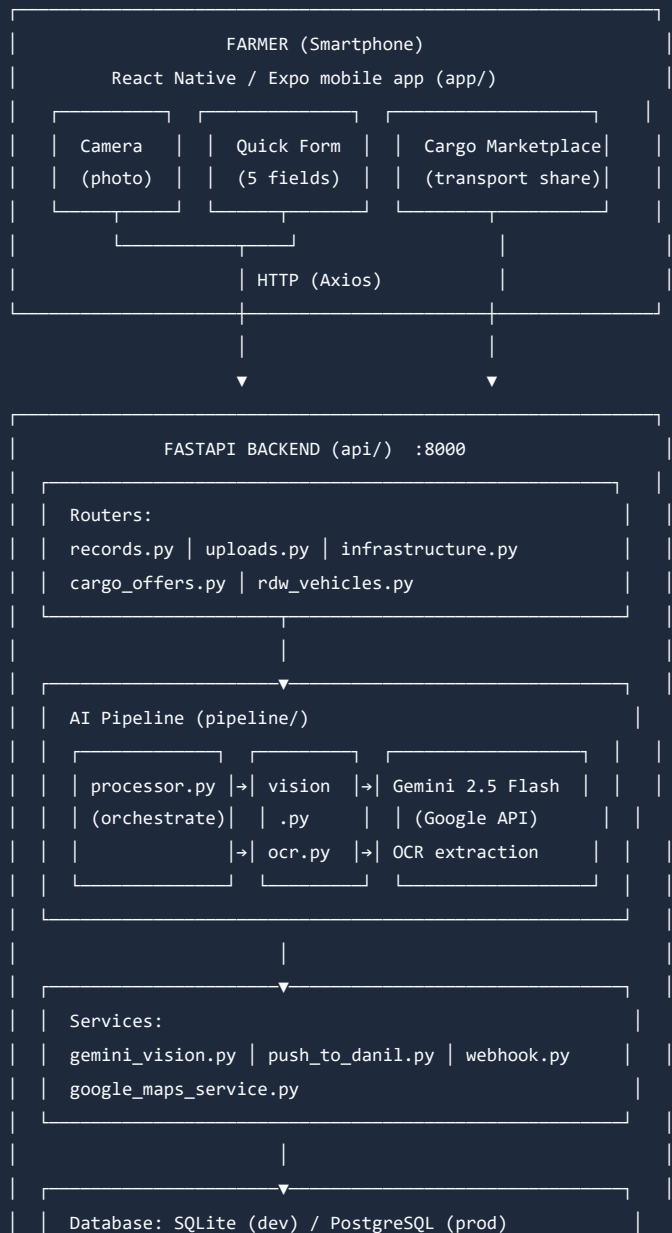
- └ Cargo offers overview with stats
- └ Active offers table with cancel option

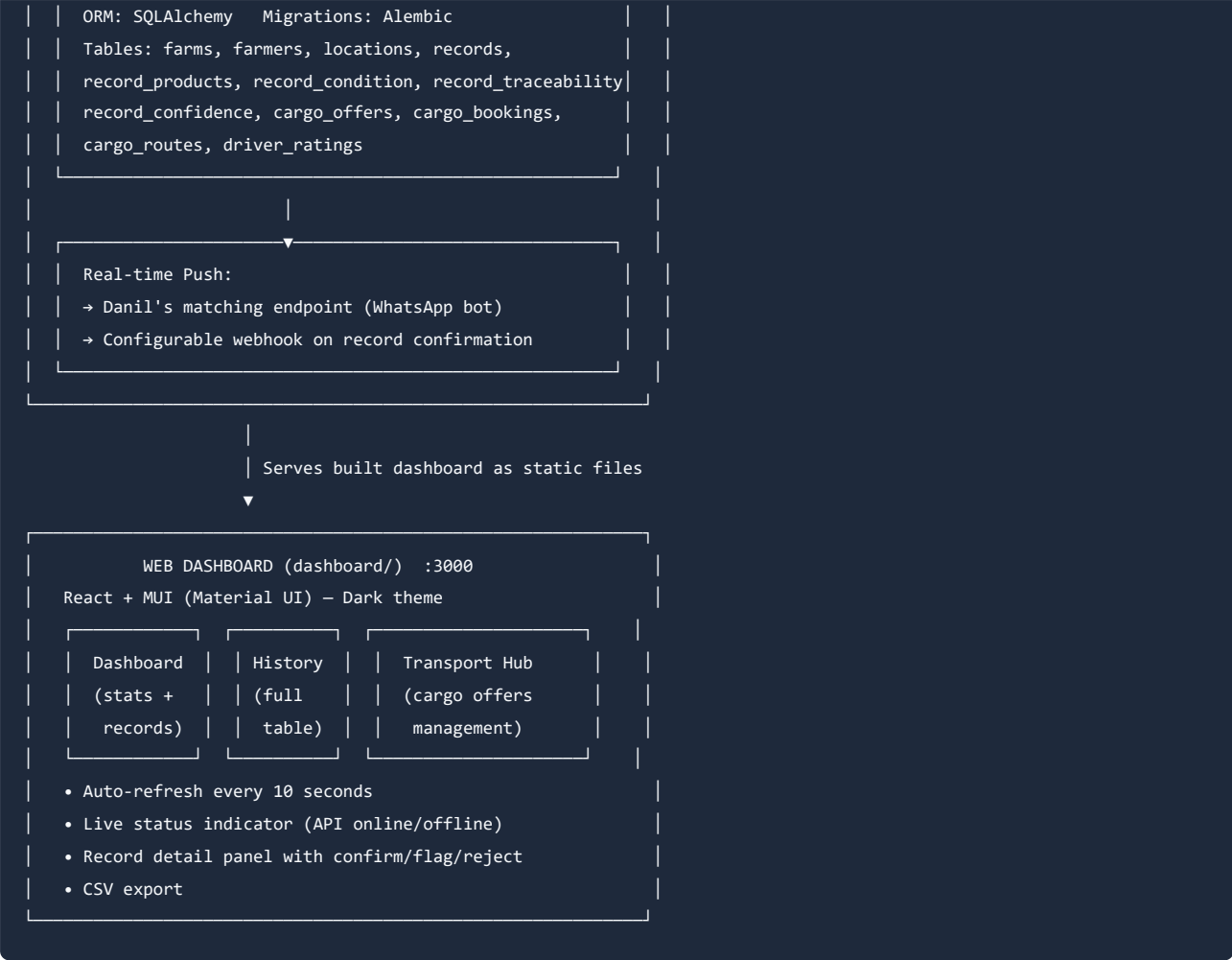
4. History tab – Full filterable record history

5. CSV Export – Download filtered records as spreadsheet

3. Technical Design

3.1 Architecture Overview





3.2 Key Technical Choices

Decision	Choice	Rationale
Mobile framework	React Native (Expo SDK 54)	Single codebase for iOS + Android; Expo simplifies camera, location, offline storage. No native build required for dev (Expo Go).
Backend framework	FastAPI (Python 3.11)	Async-capable, auto-generates OpenAPI docs at <code>/docs</code> , lightweight, strong Pydantic validation.
AI vision model	Google Gemini 2.5 Flash	Fast multimodal model; no training required; structured JSON output for product identification. Cost-effective for MVP.
OCR	Custom regex-based extraction from Gemini output	Kept simple — Gemini handles both vision and text extraction in a single call, avoiding a separate OCR library.
Database (dev)	SQLite	Zero-config, file-based — ideal for rapid development and demos.
Database (prod)	PostgreSQL 15	Production-grade, relational, good JSON support, mature ecosystem.

Decision	Choice	Rationale
ORM	SQLAlchemy 2.0	Industry standard for Python; works with both SQLite and PostgreSQL.
Object storage	Local filesystem (<code>uploads/</code>) / MinIO (Docker dev)	Simple for MVP. MinIO is S3-compatible for future cloud migration.
Dashboard	React 18 + MUI 5	Component-rich, dark-mode theme, familiar ecosystem, quick to build.
Real-time push	Fire-and-forget <code>asyncio.create_task()</code> via <code>httpx</code>	Non-blocking push to partner endpoint after every submission. Never blocks the API response.
Navigation (mobile)	React Navigation 6 (Stack + Bottom Tabs)	Standard for React Native; bottom tab navigation is farmer-friendly.
Containerisation	Docker + Docker Compose	Simple deployment without heavy DevOps; separate dev and prod compose files.

3.3 Known Limitations, Bugs & Technical Debt

[!WARNING] The following items should be addressed before the system is used in a real production scenario.

Critical

- No authentication system** — The API has no JWT/OAuth authentication. Login is by username lookup only (`farmers.username`). Any request to the API is accepted without an auth token. CORS is set to `allow_origins=["*"]`.
- Hardcoded credentials in source code** — The Danil push token is hardcoded in `push_to_danil.py`. The Gemini API key is in `api/.env` which may accidentally be committed.
- SQLite in local development** — The local dev setup uses SQLite (`farmer_data.db`), which does not support concurrent writes. The Docker prod setup correctly uses PostgreSQL.

Important

- No database migrations in practice** — While Alembic is listed as a dependency, the actual migration files are not present. Tables are created via `models.Base.metadata.create_all()` in the seed script. Adding or changing columns requires manual migration or database recreation.
- OCR is minimal** — The `ocr.py` module returns mock/placeholder data. Real OCR extraction relies entirely on Gemini's multimodal capabilities via the vision prompt. A dedicated OCR library (EasyOCR, Tesseract) was never integrated.

6. **Webhook URL is placeholder** — `webhook.py` defaults to `https://webhook.site/your-uuid` — this needs to be replaced with an actual endpoint.
7. **Duplicate category mapping logic** — Product category inference (fruit, vegetables, dairy, etc.) is duplicated in at least 3 places: `records.py`, `processor.py`, and `push_to_danil.py`. This should be refactored into a shared utility.
8. **No rate limiting** — The API has no rate limiting on any endpoint.
9. **No automated tests in CI** — While a `tests/` directory exists and some test scripts (`test_submit.py`, `test_danil_push.py`) are present, there is no test suite or CI pipeline.

Minor / Nice-to-have

10. **Settings tab is placeholder** — The dashboard's "Settings" tab shows "Settings — coming soon" with a gear emoji.
11. **Offline queue (mobile)** — Implemented but lightly tested. The `offlineQueue.js` stores submissions locally when offline and syncs when reconnected.
12. **No image compression before upload** — Photos are uploaded at full resolution. The docs mention compression to ≤ 1.5 MB, but this is not implemented.
13. **No push notifications** — Farmers don't receive notifications when records are confirmed or flagged.
14. **Cargo booking flow incomplete** — `CargoBookingScreen.js` exists but the full end-to-end booking and rating flow is not fully connected.

3.4 Recommendations for Maintenance and Retraining

Maintenance

1. **Set up proper authentication** — Implement JWT-based auth with token refresh. FastAPI's `Depends()` pattern makes this straightforward.
2. **Add Alembic migrations** — Run `alembic init alembic` and generate migration scripts for all existing models. This is essential before any schema changes.
3. **Move secrets to environment** — Move the Danil push token to `.env` and reference it via `os.getenv()`. Ensure `.env` is in `.gitignore`.
4. **Consolidate category mapping** — Create a single `utils/category_mapper.py` function and import it everywhere.
5. **Set up monitoring** — Add Sentry for error tracking (both API and mobile app). The docs reference this but it's not implemented.
6. **Database backups** — Set up automated daily backups of the PostgreSQL database, especially before the pilot.

AI Model Retraining / Tuning

7. **Gemini prompt refinement** — The current prompt in `geminivision.py` (lines 74–82) can be improved with few-shot examples of expected JSON output and farm-specific product types.
 8. **Confidence threshold tuning** — The auto-confirm threshold is `overall >= 0.7` (in `processor.py` line 125). After gathering real pilot data, analyse accuracy and adjust.
 9. **Consider a fine-tuned model** — For production at scale, train a YOLOv8 model on farm-specific produce images. This would reduce API costs and improve speed.
 10. **Build a validation dataset** — Collect ≥50 labelled test images (as specified in `docs/FARMER_DATA_SYSTEM_README.md` Section 11) and measure per-field accuracy.
-

Deliverable 2 — Reflection

2a. Tool Reflection — Where to Go from Here

What Works Well

- **The core submission flow is solid.** A farmer can open the app, take a photo, see AI-prefilled fields, and submit a record in under a minute. The 5-field form is genuinely minimal.
- **Gemini integration delivers.** The Google Gemini 2.5 Flash model provides reasonable product identification, quantity estimation, and condition assessment from a single photo — without any custom model training.
- **The dashboard auto-refreshes** every 10 seconds and shows new submissions in real-time with toast notifications. The dark-themed MUI design is professional.
- **Push-to-Danil integration** works reliably as a fire-and-forget background task — partner systems see records within ~1 second of submission.
- **Cargo Marketplace** is a differentiated feature that no comparable system offers. The RDW vehicle lookup, 20% cargo reduction, and Waze/Google Maps deep links are well-implemented.
- **Accessibility and responsive design** are well-documented and implemented (WCAG 2.1 AA compliance, 320px–1024px responsive breakpoints, screen reader support).

What Does Not Work as Intended

- **Authentication is absent** — This is the single biggest gap. Without proper auth, the system cannot be deployed for real users.
- **OCR is effectively a placeholder** — The dedicated OCR module returns mock data. All text extraction depends on Gemini's multimodal response, which is less reliable for small label text.
- **Offline mode is lightly tested** — While the architecture supports offline queuing, edge cases (photo uploads while offline, sync conflicts) have not been thoroughly validated.
- **Cargo booking end-to-end** — Creating and browsing offers works, but the booking confirmation → pickup → delivery lifecycle is not fully connected.

If We Had More Time / Budget / Resources

Priority	Enhancement	Effort estimate
P0	Implement JWT authentication & role-based access	1–2 weeks
P0	Add proper database migrations (Alembic)	2–3 days
P1	Integrate dedicated OCR (EasyOCR) for label/expiry extraction	1 week
P1	Complete cargo booking lifecycle	1–2 weeks
P1	Add push notifications (Firebase Cloud Messaging)	1 week
P2	Build validation dataset & accuracy reporting	2 weeks
P2	Add multi-language support to dashboard	1 week
P3	Train custom YOLOv8 model on farm-specific images	3–4 weeks
P3	Transport leg tracking (loaded → in transit → delivered)	2–3 weeks

How Should the Tool Evolve (6–12 Months)

Near-term (0–3 months):

- Authentication and security hardening — this is the gate to any real pilot
- Validation dataset collection with real farm photos during a pilot
- Confidence threshold tuning based on pilot accuracy data

Medium-term (3–6 months):

- Yield prediction by combining quantity records over time
- Buyer-facing view showing aggregate product availability
- Integration with other SFSC projects (visual yield estimation, route optimisation)
- Push notifications for farmers when records are confirmed/flagged

Long-term (6–12 months):

- Fine-tuned vision model trained on farm-specific imagery
- Freshness and shelf-life estimation from visual cues
- Multi-product submissions (log several products in one session)
- Full transport chain tracking: farm → truck → hub → buyer

2b. Personal Process Reflection

What I Learned

Technically:

- Building a full-stack system from mobile app to AI pipeline to dashboard — and understanding how all the pieces connect through APIs and data schemas.
- Working with multimodal AI (Gemini) for structured data extraction from photos, including prompt engineering, confidence scoring, and graceful fallback handling.
- React Native with Expo for cross-platform mobile development, including camera integration, offline storage, and accessibility features.
- Docker containerisation for deployment and the differences between development and production configurations.

Professionally:

- Scoping an MVP: the hardest part is deciding what *not* to build. The spec document (FARMER_DATA_SYSTEM_README) describes a 10–12 week plan — delivering the most impactful features first required constant prioritisation.
- Writing documentation that someone else can actually use. The handover requirement forced me to think about every assumption I was making.
- Working with partner systems (Danil's WhatsApp bot) — designing an integration that's resilient to failures (fire-and-forget with logging) rather than tightly coupled.

Personally:

- Gained confidence in making architectural decisions and defending them. Choosing Gemini over a custom-trained model, SQLite for dev vs PostgreSQL for prod, fire-and-forget push vs synchronous webhook — each required weighing trade-offs.
- Learned to balance perfectionism with pragmatism. Shipping a working system with known limitations (documented) is more valuable than an unfinished perfect system.

What Surprised Me

- **How effective Gemini is out-of-the-box** for produce identification. With a well-crafted prompt, it correctly identifies product types and estimates quantities from crate photos without any fine-tuning.
- **How much work goes into the "boring" parts** — form validation, error handling, offline support, accessibility — these account for far more development time than the AI pipeline.
- **How important the UX is for adoption.** Farmers won't use a tool that requires more than 3 taps. Every extra field or screen is a potential drop-off point.

What Was Harder / Easier Than Expected

	Aspect	Notes
Harder	Accessibility compliance	Meeting WCAG 2.1 AA across all screens, with screen reader support and proper touch targets, was more meticulous work than anticipated.
Harder	Offline-first architecture	Handling photo uploads, queue sync, and conflict resolution when connectivity is intermittent is a significant engineering challenge.
Easier	Gemini API integration	The API is straightforward, the documentation is good, and the model handles agricultural images well out of the box.

	Aspect	Notes
Easier	Dashboard development	MUI provides so many pre-built components that building a professional dashboard took days rather than weeks.

How I Grew During This Project

- From "I can write code" to "I can architect a system" — understanding how to break a complex problem into components (mobile app, API, AI pipeline, dashboard, integrations) and define the interfaces between them.
- From "it works on my machine" to "it's deployable" — learning Docker, environment configuration, and the difference between a demo and a production system.
- From building features to delivering value — understanding that the farmer's experience matters more than the technical elegance of the AI pipeline.

2c. Curriculum & Educational Value

Could This Type of Project Be Part of the DS&AI Curriculum?

Yes, strongly. This project sits at the intersection of multiple Data Science & AI competencies:

Competency	How this project exercises it
Machine Learning / AI	Multimodal AI for structured data extraction, confidence scoring, prompt engineering
Software Engineering	Full-stack development (mobile, API, dashboard), REST API design, database modelling
Data Engineering	Data pipeline design (photo → AI → structured record → push to partner), schema design, ETL
UX / Human-Centred Design	Designing for low-tech-literacy users, accessibility, user testing
DevOps	Docker, deployment, environment configuration, CI/CD (potential)
Ethics & Privacy	Handling farmer data responsibly, GDPR considerations, data minimisation

Recommended stage: Year 3 or as a graduation project. It requires a foundation in Python, web development, and ML, but the real learning comes from integrating all of these into a working system for real users.

What I Enjoyed

- The freedom to make architectural decisions and own the full stack — from choosing Gemini over GPT-4o Vision to designing the database schema.

- Working with real stakeholders (farms, logistics hubs) and understanding how technology serves a practical need.
- Seeing the system work end-to-end for the first time: a photo of tomato crates becoming a structured logistics record in the dashboard within seconds.
- The cargo marketplace feature — it was the most creative part, solving a real coordination problem between farmers.

What I'd Recommend Keeping for Future Student Assistants

- **Real client engagement** — working with LCB and actual farms grounds the project in reality.
- **Full-stack ownership** — the breadth of the project (mobile, API, AI, dashboard) forces you to learn outside your comfort zone.
- **Handover documentation requirement** — it's painful but enormously educational. It forces you to think about someone else reading your code.
- **Integration with partner projects** — collaborating with Danil's WhatsApp bot team mirrors real-world software development.

What Could Be Organised or Supported Differently

- **Earlier access to real test data** — getting actual farm produce photos earlier in the project would have allowed better AI tuning and validation.
- **Clearer deployment infrastructure from the start** — knowing the target server specs, SSH access, and network constraints earlier would have avoided deployment surprises.
- **Mid-point code review** — a technical review at the midpoint of the project (e.g., week 6) from an experienced developer would have caught architectural issues (like the missing auth) earlier.
- **Structured user testing sessions** — while the user testing guide was written, organising actual sessions with farmers requires coordination from BUAs/LCB that should be planned in advance.
- **Budget for cloud services** — using free-tier APIs (Gemini) and local storage works for MVP, but a small cloud budget (e.g., for Google Cloud, AWS S3, or a CI/CD pipeline) would have enabled a more production-ready system.

Appendix: Quick Command Reference

Task	Command
Start everything (Windows)	<code>start.bat</code>
Start API (manual)	<code>cd api && python -m uvicorn app.main:app --reload --host 0.0.0.0 --port 8000</code>
Start Dashboard (dev)	<code>cd dashboard && npm start</code>
Start Mobile App	<code>cd app && npx expo start</code>
Deploy production	<code>docker-compose -f docker/docker-compose.prod.yml up -d --build</code>

Task	Command
View API docs	Open <code>http://localhost:8000/docs</code>
Seed database	<code>cd api && python seed.py</code>
Reset database	<code>cd api && python reset_db.py</code>
View production logs	<code>docker-compose -f docker/docker-compose.prod.yml logs -f api</code>
Stop production	<code>docker-compose -f docker/docker-compose.prod.yml down</code>
Build mobile (Android)	<code>cd app && eas build --platform android</code>
Build mobile (iOS)	<code>cd app && eas build --platform ios</code>
Export records (CSV)	<code>curl http://localhost:8000/v1/records/export > records.csv</code>

Appendix: Test User Accounts

The seed script creates the following test users:

Username	Farm	Region
<code>henk_van_den_berg</code>	De Walhoeve	Goirle
<code>maria_jansen</code>	—	—
<code>peter_de_vries</code>	De Dobbelhoeve	Udenhout
<code>kees_van_der_meer</code>	—	—
<code>anneke_vink</code>	—	—
<code>henk_smeets</code>	—	—

Additional seed data includes a Central Hub in Udenhout (Brabants Streekgoed & The Food Directors) with receiving bay and cold storage locations.

Document prepared: June 2026

Last updated: June 26, 2026